AI Engineer Hackathon Assignment

Role: AI Engineer, Generative AI & Machine Learning

Company: Mindbowser

Location: Pune, India, Baner

Assignment Type: Take-home hackathon followed by offline technical presentation

Estimated Effort: 6-8 hours

Submission Window: 3-4 days from assignment date

Assignment Format

This assignment is designed as a practical hackathon-style challenge.

The candidate is expected to independently design, develop, test, and document a working healthcare-focused AI application at their own end. The goal is to evaluate the candidate's ability to build a real, runnable AI solution rather than only explain concepts theoretically.

The candidate may use open-source tools, public datasets, synthetic healthcare documents, LLM APIs, local LLMs, or cloud services, provided that no real patient data or confidential information is used.

Shortlisting and Offline Presentation

After the candidate submits the assignment, Mindbowser's evaluation team will review the implementation.

If the assignment is shortlisted, the candidate will be invited to Mindbowser's office in Baner, Pune, for an offline technical presentation and live application demo in front of the evaluator panel.

During the offline evaluation, the candidate must present:

- The working application

- The architecture and design approach
- The RAG pipeline and LLM integration
- Prompt engineering strategy
- Agent/tool workflow implementation
- API/demo flow
- Key technical decisions and trade-offs
- Limitations and future improvements

The application should be runnable during the offline presentation. The candidate should be prepared to run the project locally or through a hosted/demo environment.

Offline Demo Expectations

During the in-office presentation, the evaluator panel may ask the candidate to:

- Run the application live
- Ingest documents again
- Ask new unseen healthcare-related questions
- Show how unknown answers are handled
- Explain source citations
- Walk through important parts of the code
- Explain model, embedding, and vector database choices
- Demonstrate Docker setup, if available
- Modify a prompt or configuration during the session
- Explain how the solution would scale in production
- Discuss security, PHI, and healthcare compliance considerations

The candidate should be able to explain their own implementation clearly and confidently.

Objective

Build a small healthcare-focused AI assistant that can answer user questions based on a given set of healthcare documents using a Retrieval-Augmented Generation pipeline.

The solution should demonstrate practical understanding of:

- RAG pipelines
- LLM integration
- Prompt engineering
- Vector databases or semantic search
- Basic agentic/tool-based workflow
- API development
- Docker-based deployment
- Clean coding and documentation

Problem Statement

Mindbowser works with healthcare clients who often need AI assistants that can answer questions from internal clinical, operational, or compliance documents.

You need to build a prototype AI assistant that answers questions using only the provided healthcare knowledge base.

The assistant should:

1. Ingest healthcare documents.
2. Store searchable embeddings in a vector database.
3. Retrieve relevant context for a user question.
4. Generate a grounded answer using an LLM.
5. Provide source references or citations.
6. Avoid hallucinating when information is not present in the documents.
7. Expose the functionality through an API.
8. Include basic logging and error handling.

Dataset

Use any small set of healthcare documents. You may create synthetic documents yourself.

Suggested document topics:

- Patient discharge instructions
- Appointment scheduling policy
- Insurance eligibility FAQ
- HIPAA/privacy guidelines
- Medication refill policy
- Telehealth consultation guidelines

Important: Do not use real patient data or PHI.

The candidate should include the documents inside a `/data` folder.

Functional Requirements

1. Document Ingestion

Create a script or API endpoint that:

- Read documents from a folder.
- Splits documents into chunks.
- Generates embeddings.
- Stores embeddings in a vector database.

Preferred tools:

- FAISS
- ChromaDB
- Weaviate
- Pinecone, if already familiar

---

2. RAG-based Question Answering

Create an API endpoint:

```
POST /ask
```

Sample request:

```
{
  "question": "Can a patient request a medication refill through telehealth?"
}
```

Sample response:

```
{
  "answer": "Yes, patients can request medication refills through telehealth if the medication is already prescribed and does not require an in-person evaluation.",
  "sources": [
    {
      "document": "telehealth_policy.pdf",
      "chunk": "Medication refill requests may be reviewed during telehealth visits..."
    }
  ],
  "confidence": "medium"
}
```

The answer must be generated only from retrieved context.

If the answer is not available in the documents, the assistant should respond clearly:

```
I could not find this information in the provided documents.
```

---

3. LLM Integration

Use any one of the following:

- Open-source local LLM through Ollama, such as Llama or Mistral
- Hugging Face model
- OpenAI-compatible API
- Any other LLM provider

Bonus points for using an on-premise/local LLM such as:

- Llama
- Mistral
- Phi
- Gemma

---

4. Prompt Engineering

Design a prompt that ensures:

- The assistant answers only from provided context.
- The assistant refuses to guess.
- The assistant keeps responses clear and professional.
- The assistant avoids giving direct medical diagnosis or unsafe medical advice.

Include the final prompt in the README.

---

5. Basic Agentic Workflow

Implement one simple tool-based or agentic workflow.

Example:

If a question is about appointment booking, route it to a mock tool:

```
check_available_slots(department, date)
```

If the question is about document knowledge, use RAG.

The workflow can be implemented using:

- LangChain
- CrewAI
- AutoGen
- LlamaIndex
- Custom simple router logic

Expected behavior:

```
Question: "Can I book a cardiology appointment for Monday?"
```

```
Assistant: "I can check mock appointment availability. Available slots are..."
```

This does not need to connect to a real appointment system.

---

6. API Development

Build a backend API using one of:

- FastAPI
- Flask
- Django REST Framework
- Node.js/Express

Required endpoints:

```
POST /ingest
POST /ask
GET /health
```

---

7. Dockerization

Provide a `Dockerfile`.

Bonus points for `docker-compose.yml`.

The application should be runnable with clear instructions.

Non-Functional Requirements

The solution should demonstrate:

- Clean project structure
- Readable code
- Proper error handling
- Logging for important steps
- Basic configuration through `.env` or config file
- No hardcoded secrets
- Clear README

- Reasonable response time for small datasets

Expected Project Structure

```
healthcare-ai-assistant/
  app/
    main.py
    rag.py
    embeddings.py
    llm.py
    agent.py
    config.py
  data/
    document_1.txt
    document_2.txt
  vector_store/
  tests/
  requirements.txt
  Dockerfile
  docker-compose.yml
  README.md
```

Required Deliverables

The candidate should submit:

1. GitHub repository link or zipped source code
2. README with setup and run instructions
3. Architecture explanation
4. API examples using curl/Postman
5. Sample questions and responses
6. Dataset/source details
7. Dockerfile, if implemented
8. Short explanation of:
    - LLM used
    - Embedding model used
    - Vector database used
    - Prompting strategy
    - Agent/tool workflow
    - Limitations and future improvements

Optional:

- Demo video
- Unit tests
- Evaluation script
- Simple frontend UI
- Docker Compose setup

Evaluation Criteria

Stage 1: Assignment Review

Mindbowser will review the submitted code and documentation based on:

- Completeness of implementation
- RAG pipeline quality
- LLM and prompt engineering approach
- Code structure and readability
- API design
- Error handling and logging
- Documentation quality
- Deployment readiness

Stage 2: Offline Technical Presentation

Shortlisted candidates will be evaluated in person based on:

- Ability to run the application live
- Clarity of technical explanation
- Understanding of their own code
- Ability to answer follow-up questions
- Debugging and problem-solving approach
- Practical production thinking
- Healthcare data privacy awareness

Suggested Public Data Sources

Candidates may use one or more of the following public or synthetic healthcare data sources. They should not use real patient data, PHI, or any confidential client data.

Source	Best Use in Assignment	Link
MedlinePlus Health Topics	Consumer-friendly healthcare articles for RAG knowledge base	MedlinePlus XML Files
WHO Fact Sheets	Disease, public health, and medical topic summaries	WHO Fact Sheets
CDC Open Data	Public health datasets, disease statistics, vaccination, chronic disease data	CDC Open Data
CDC PLACES	Location-based health outcomes, preventive care, risk behavior data	CDC PLACES Data Portal
ClinicalTrials.gov	Clinical trial search, eligibility, recruitment status, study information	ClinicalTrials.gov API
openFDA	Drug labels, adverse events, recalls, device data	openFDA
CMS Public Data	Medicare providers, utilization, payments, provider directories	CMS Data Available to Everyone
AHRQ MEPS	Healthcare cost, utilization, insurance, expenditure data	AHRQ MEPS
AHRQ HCUPnet	Hospital utilization and healthcare statistics	HCUPnet
PubMed Central Open Access	Research articles for advanced RAG use cases	PMC Open Access Subset

Synthea	Synthetic patient records, FHIR data, mock EHR workflows	Synthea
MedQuAD	Medical question-answer pairs for QA testing or evaluation	MedQuAD GitHub
WHO Global Health Observatory	Global health indicators and structured API-based data	WHO GHO
India Open Government Data	Public Indian government datasets, including health-related datasets	data.gov.in

Candidate Instruction Note

- This is a hackathon-style assignment. We are not expecting a perfect production product, but we do expect a working prototype that demonstrates strong AI engineering fundamentals.
- Please ensure the application is runnable before submission and before the offline presentation. Do not use real patient data, PHI, or confidential healthcare records. Use only synthetic or public healthcare data.